

KNEO AGENT SDK

KNEO AGENT SDK

API Reference Manual

Complete reference for the public Kneo Agent SDK surface, including runtimes, workflows, providers, middleware, utilities, and embedded practical guides.

VERSION

v2.0.0

RUNTIME

Python 3.12+

LICENSE

MIT

Prepared as a printable reference edition with API details, usage guidance, and architecture-facing concepts consolidated into one manual.

asyncio

fully typed

agents + workflows

Table of Contents

Use this section as the printable navigation guide for the manual. The linked entries mirror the document structure that follows after the introduction.

OVERVIEW

[Introduction](#)

[Design Patterns](#)

[Quick Start](#)

CORE API

[Agent](#)

[run\(\)](#)

[stream\(\)](#)

[chat\(\)](#)

[history methods](#)

[as_tool\(\)](#)

[AgentBuilder](#)

[Middleware APIs](#)

[Production middleware](#)

[Observability](#)

[OpenTelemetryMiddleware](#)

WORKFLOWS

Workflow

WorkflowEdge

WorkflowComponent

Built-In Orchestration Runtimes

SequentialWorkflow

ConcurrentWorkflow

HandoffWorkflow

GroupChatWorkflow

RetryStep

WorkflowBuilder

DATA TYPES

AgentConfig

RunConfig

RunResult

StreamChunk

Message

Tool Types

AgentRuntime

PATTERNS AND RUNTIME

BridgeAgentFactory

NativeRuntimeFactory

AdapterAgentFactory

Bridge Executors

Platform Adapters

RuntimeImpl Protocol

PROVIDERS AND UTILITIES

Providers

Skill APIs

ToolRegistry

Secret Providers

MCP

Messages Utilities

Logging Utilities

REFERENCE

Exceptions

Custom Provider

Streaming Guide

Bridge vs Adapter

GUIDES

Guides Overview

Agent Middleware

Agent Skills

MCP

Human In The Loop

API Reference Manual

Complete reference for all public classes, methods, protocols, and utilities.

v2.0.0

Python 3.12+

MIT License

asyncio

fully typed

What's new in v2.0.0

Released **2026-06-03**. A **major** release — the first behaviour-breaking cut since 1.0. It bundles the deferred `RunConfig` / `middleware` / `adapter` break-set with the post-1.5.0 audit fixes, so callers adapt once. **Read the [migration guide](#) before upgrading** — `pip install --upgrade` may change behaviour without a code change. Full detail in the [CHANGELOG](#).

Changed — breaking BREAKING

`RetryMiddleware.retry_tool_calls` now defaults to `False` (was `True`). Tool calls are not retried unless you opt in, so non-idempotent tools aren't silently re-executed on a transient failure. Pass `retry_tool_calls=True` to restore the prior behaviour. Model-call retries are unchanged.

A partial `RunConfig` override now merges with agent / skill defaults instead of replacing `max_iterations` / `temperature` / `tools` wholesale — fields you don't set keep the agent's configured value. Set a field explicitly to override it. Merged agent / skill defaults are now revalidated, so a bad value raises `ValueError` at build time (a non-numeric value raises `ValueError`, not `TypeError`).

OpenAI Agents `Adapter` call interface aligned to the `openai-agents` `Runner` — it now calls `Runner.run(starting_agent=...)` / `run_streamed(...).stream_events()` (was an invented `run(agent=...)` / `stream()` shape) and reads object- or dict-shaped results. Update a custom runner that matched the old shape. The Google ADK and LangChain adapters are **pass-through** (no per-call `RunConfig` injection — the wrapped object owns its config); `temperature` is uninjected on every adapter. These remain *demonstration* adapters — for a production OpenAI path use the native runtime (`NativeRuntimeFactory.for_openai`).

`Workflow.stream()` raises `StreamingNotSupportedError` on `ConcurrentWorkflow` / `HandoffWorkflow` / `GroupChatWorkflow` / the graph `Workflow` — they cannot stream incrementally, and `supports_streaming()` already reports `False`. `SequentialWorkflow.stream()` now raises `StreamingNotSupportedError` (not `NotImplementedError`) when a participant can't stream.

Fixed FIXED

ADK Bridge no longer crashes on an empty `candidates` list — Gemini safety blocks / `MAX_TOKENS` - with-no-content / recitation stops return `{"candidates": []}`, which raised `IndexError` (also through `stream_tokens`); it now degrades to empty text.

OpenAI Agents *Adapter* reads object- or dict-shaped results/usage — normalisation for `final_output` / `new_items` / `usage` (including object-shaped `prompt_tokens` / `completion_tokens` aliases) and the nested `response.output_text.delta` stream shape.

LangChain adapter preserves conversation history on non-alternating roles (a leading assistant turn no longer drops the whole `chat_history`).

Bridge usage backfills `total_tokens` so a custom `RuntimeImpl` reporting only input / output no longer yields `total_tokens=0`; native Google ADK OTel `gen_ai.system` normalises to the GenAI enum; duplicate workflow participant / executor names are detected instead of silently collapsing.

Docs & examples [docs](#)

New migration guide (`docs/user/upgrading_to_2.0.md`); the API reference documents `for_openai` `base_url` / `api_key`, the `human_step` `metadata` `kward`, and the `HumanInTheLoopStep` `description` / `metadata` fields.

README-dev package layout adds the `middleware/` subpackage, `workflows/retry.py`, and `utils/secrets.py`; the architecture-summary workflow matrix adds `RetryStep`; stale post-1.4 / 1.5 usage comments are reworded.

Introduction

Kneo Agent is a framework-agnostic Python Agent SDK that lets you build LLM-powered agents and composite workflows running on Google ADK, OpenAI Agents SDK, or LangChain using four runtime categories: **Bridge**, **Native**, **Adapter**, and **Workflow**.

The entire public API is exported from the top-level `kneo_agent` package. Most application code only needs three imports:

```
from kneo_agent import Agent, AgentBuilder, RunConfig, ToolDefinition
from kneo_agent.patterns import NativeRuntimeFactory
from kneo_agent.utils import ToolRegistry
```

Design Patterns [CONCEPT](#)

Bridge Pattern

Decouples the agentic loop *strategy* (Simple, ReAct, Plan-Act) from the platform *implementation* (Google ADK, OpenAI, LangChain). Both sides vary independently. Use when designing from scratch.

Adapter Pattern

Wraps an existing, fixed-interface platform object and translates its API to `AgentRuntime`. The platform owns its own loop. Use when integrating an existing agent executor.

Native Runtime

Hands loop ownership to the platform SDK/runtime itself while still exposing `AgentRuntime`. Use for platform-owned runtimes such as the OpenAI Agents SDK and native Google ADK execution.

Workflow Runtime

Composes agents, nested workflows, and custom function steps using the composite pattern. A workflow itself implements `AgentRuntime`, so it can be used as an agent runtime directly or wrapped with `workflow.as_agent()`.

Quick Start

Two convenience builders cover the common cases. `build_sync_agent` is the friction-free starting point — a blocking wrapper that hides `asyncio` entirely, ideal for scripts and prototypes. `build_agent` is the async equivalent for code already inside an event loop; it accepts the same parameters and returns a regular `Agent` you `await`. The parameter table below applies to both.

```
build_sync_agent(provider: "openai" | "langchain" | "google-adk", **kwargs) → SyncAgent
```

Blocking one-liner from `kneo_agent.simple`, re-exported as `kneo_agent.build_sync_agent`. Picks a runtime factory based on `provider`, applies sensible defaults, attaches optional tools and middlewares, and returns a `SyncAgent` whose `run`, `chat`, and `stream` methods block until the underlying coroutine completes — no `asyncio` at the call site.

```

from kneo_agent import build_sync_agent

agent = build_sync_agent(
    "openai",
    model="gpt-4o-mini",
    system_prompt="You are a helpful assistant.",
)

print(agent.chat("What is 2 + 2?"))    # blocks, returns str

```

Heads up: Each call spins up a fresh event loop via `asyncio.run`. `SyncAgent` raises `RuntimeError` if invoked from inside an already-running event loop (Jupyter, FastAPI handlers, other async code) rather than silently deadlocking; reach for the async `build_agent` instead in those contexts.

PARAMETER	TYPE	DESCRIPTION
<code>provider</code>	<code>"openai" "langchain" "google-adk" None</code>	Provider shortcut. Mutually exclusive with <code>runtime</code> .
<code>runtime</code>	<code>AgentRuntime None</code>	Pre-built runtime. Use this when you already have a custom runtime, a workflow, or a third-party adapter.
<code>model</code>	<code>str</code>	OpenAI model id. Default <code>"gpt-4o-mini"</code> .
<code>openai_client</code>	<code>Any None</code>	Optional <code>openai.AsyncOpenAI</code> client.
<code>chat_model</code>	<code>Any</code>	Required for <code>"langchain"</code> : any <code>langchain_core.language_models.BaseChatModel</code> .
<code>adk_runner</code>	<code>Any</code>	Required for <code>"google-adk"</code> : an ADK runner with <code>run_async</code> .

<code>adk_app_name / adk_user_id / adk_session_id</code>	<code>str</code>	ADK identifiers. All default to "default" .
<code>name / description / system_prompt</code>	<code>str / str / str None</code>	Agent metadata.
<code>tools</code>	<code>ToolRegistry None</code>	Tool registry to attach. For LangChain Bridge runtimes the registry's handlers are also wired into the runtime for dispatch.
<code>middlewares</code>	<code>list[AgentMiddleware] None</code>	Middlewares to attach (e.g. <code>OpenTelemetryMiddleware()</code>).
<code>strategy</code>	<code>"simple" "react" "plan-act"</code>	Bridge agent loop strategy. Default "react" .

`build_agent(*args, **kwargs)` → Agent

Async variant of `build_sync_agent` . Accepts the same parameters (see the table above) but returns a regular async `Agent` directly, with no `SyncAgent` wrapper. Use this when you are already writing async code — e.g. inside a Jupyter cell, a FastAPI handler, or another `asyncio` task.

```
import asyncio
from kneo_agent import build_agent

agent = build_agent(
    "openai",
    model="gpt-4o-mini",
    system_prompt="You are a helpful assistant.",
)

async def main():
    print(await agent.chat("What is 2 + 2?"))

asyncio.run(main())
```

class SyncAgent [wrapper](#)

Composition wrapper returned by `build_sync_agent`. Properties (`config` , `agent_name` , `runtime_name` , `history`) and history-management methods (`clear_history` , `inject_history`) pass through unchanged. The async surface (`run` , `chat` , `stream`) is replaced with blocking equivalents; `stream` returns the full list of chunks rather than yielding incrementally.

Explicit builder

Use the explicit chain when you need behavior `build_agent` does not expose: workflow composition, custom `RuntimeImpl` implementations, fine-grained per-step skill loading, etc.

```

import asyncio
from kneo_agent import AgentBuilder, ToolDefinition
from kneo_agent.patterns import NativeRuntimeFactory
from kneo_agent.workflows import WorkflowBuilder
from kneo_agent.utils import ToolRegistry

# 1. Define tools
registry = ToolRegistry()

@registry.tool(
    name="get_weather",
    description="Get current weather for a city.",
    parameters={"type": "object", "properties": {"city": {"type": "string"}}},
    "required": ["city"]},
)
def get_weather(args: dict) -> str:
    return f"22 °C and sunny in {args['city']}"

# 2. Create a platform-owned runtime
runtime = NativeRuntimeFactory.for_openai(model="gpt-4o", strategy="react")

# 3. Build the agent
agent = (
    AgentBuilder()
    .with_name("Weather Assistant")
    .with_system_prompt("You are a helpful weather assistant.")
    .with_tools(registry.definitions)
    .use_runtime(runtime)
    .build()
)

# 4. Run
async def main():
    result = await agent.run("What is the weather in Tokyo?")
    print(result.final_message)

```

```
asyncio.run(main())
```

Workflow note: Workflows are first-class runtimes. You can compose multiple agents into a `SequentialWorkflow` and expose that workflow as a normal `Agent` with `workflow.as_agent(...)`.

Agent

The central class of the SDK. Stateful, framework-agnostic, and built via `AgentBuilder`. Never instantiated directly.

class Agent CLASS

A stateful conversational agent. Holds conversation history and delegates all LLM work to an injected `AgentRuntime`.

Properties

PROPERTY	TYPE	DESCRIPTION
<code>agent_name</code>	<code>str</code>	Alias for <code>config.name</code> .
<code>runtime_name</code>	<code>str</code>	The name of the injected runtime (e.g. <code>"react@openai-agents"</code> , <code>"react@langchain"</code> , or <code>"google-adk-native"</code>).
<code>config</code>	<code>AgentConfig</code>	Static agent configuration (read-only).
<code>history</code>	<code>list[Message]</code>	Shallow copy of the current conversation. Safe to read; cannot be mutated.

```
async def run(user_message: str, *, run_config: RunConfig | None = None,
**extra) → RunResult
```

Send a user message and execute a full agent run. Appends both the user turn and the final assistant reply to history.

PARAMETER	TYPE	DEFAULT	DESCRIPTION
<code>user_message</code>	<code>str</code>	—	The text sent as the user turn.
<code>run_config</code>	<code>RunConfig</code> <code>None</code>	<code>None</code>	Per-call overrides merged on top of the agent's defaults.
<code>**extra</code>	<code>Any</code>	—	Forwarded into <code>RunConfig.extra</code> .

RETURNS

`RunResult` — full run output including `final_message`, `iterations`, tool calls, and timing.

```
# Simple run
result = await agent.run("Who invented Python?")
print(result.final_message)
print(f"Took {result.duration_ms:.0f} ms, {result.iterations} iteration(s)")

# With per-call temperature override
result = await agent.run(
    "Write a haiku about Python.",
    run_config=RunConfig(temperature=0.9),
)
```

```
async def stream(user_message: str, *, run_config: RunConfig | None = None) →
AsyncGenerator[StreamChunk, None]
```

Stream the agent response. Returns an async generator of `StreamChunk` objects. The final chunk always has `type == "done"`.

RAISES

`StreamingNotSupportedError` if the runtime does not support streaming.

```

async for chunk in await agent.stream("Explain async/await."):
    match chunk.type:
        case "text":
            print(chunk.content, end="", flush=True)
        case "tool_call":
            print(f"\n→ calling {chunk.tool_call.name}")
        case "tool_result":
            print(f"\n← {chunk.tool_result.name}: {chunk.tool_result.result}")
        case "done":
            print() # newline after stream

```

```

async def chat(user_message: str, **kwargs) → str

```

Convenience wrapper around `run()`. Returns only the final text string. All keyword arguments are forwarded to `run()`.

```

reply = await agent.chat("What is 2 + 2?")
# → "4"

```

```

def clear_history() → None

```

Reset the conversation to a clean slate. Subsequent runs start with no prior context.

```

def inject_history(messages: list[Message]) → None

```

Seed the conversation with pre-existing messages, e.g. loaded from a database. Replaces any current history.

```

# Restore a conversation from storage
saved = load_conversation_from_db(session_id)
agent.inject_history(saved)

# Continue the conversation
reply = await agent.chat("Where were we?")

```

```
def add_tool(tool: ToolDefinition) → None
```

Dynamically register a tool after the agent has been built. The tool is added to `config.tools` and included in all subsequent runs.

```
def as_tool(*, name: str | None = None, description: str | None = None,
parameters: dict | None = None, arg_name: str = "input", run_config: RunConfig
| None = None, skills: list[Skill] | None = None, include_history: bool =
False) → AgentTool
```

Expose the agent as a first-class tool without mutating the wrapped agent's history. By default the generated tool accepts a single string argument and runs against an isolated message list.

Agent tool note: Set `include_history=True` when the delegated tool call should be seeded with the wrapped agent's current conversation history.

AgentBuilder

class AgentBuilder BUILDER

Fluent builder for `Agent` instances. Every method returns `self` for chaining. Call `.build()` last.

Note: You must call one of `use_bridge()`, `use_adapter()`, or `use_runtime()` before `build()`, or a `RuntimeNotConfiguredError` is raised.

Identity methods

METHOD	PARAMETER	DESCRIPTION
<code>.with_name(name)</code>	<code>str</code>	Set the agent's display name. Default: "unnamed-agent" .
<code>.with_description(desc)</code>	<code>str</code>	Human-readable description for documentation/registries.
<code>.with_version(ver)</code>	<code>str</code>	Semantic version string. Default: "1.0.0" .

`.with_tags(*tags)``str...`

Arbitrary labels for grouping agents (e.g. "prod" , "v2").

Behaviour methods

METHOD	PARAMETER	DESCRIPTION
<code>.with_system_prompt(p)</code>	<code>str</code>	System prompt prepended to every run.
<code>.with_tools(tools)</code>	<code>list[ToolDefinition]</code>	Replace the tool list. Passed to every <code>RunConfig</code> .
<code>.add_tool(tool)</code>	<code>ToolDefinition</code>	Append a single tool to the list.
<code>.with_tool_registry(registry, *, skill_name="tool-registry", description="...")</code>	<code>ToolRegistry</code>	Attach registry definitions and package registry handlers into an implicit skill. Useful for MCP-backed tool sets.
<code>.with_middlewares(middlewares)</code>	<code>list[AgentMiddleware]</code>	Replace the agent's static middleware chain.
<code>.add_middleware(middleware)</code>	<code>AgentMiddleware</code>	Append one middleware to the agent's static middleware chain.
<code>.with_defaults(**kw)</code>	<code>Any</code>	Override default <code>RunConfig</code> fields by name, e.g. <code>max_iterations=5</code> .

Runtime wiring methods

METHOD	PARAMETER	DESCRIPTION
<code>.use_bridge(runtime)</code>	<code>AgentRuntime</code>	Attach a Bridge-pattern runtime (any <code>AgentExecutor</code> subclass).
<code>.use_adapter(runtime)</code>	<code>AgentRuntime</code>	Attach an Adapter-pattern runtime.
<code>.use_runtime(runtime)</code>	<code>AgentRuntime</code>	Attach any object satisfying the <code>AgentRuntime</code> protocol.

```
def build() → Agent
```

Construct and return an `Agent`. Raises `RuntimeNotConfiguredError` if no runtime has been attached.

```
from kneo_agent import AgentBuilder, ToolDefinition, RunConfig
from kneo_agent.patterns import BridgeAgentFactory

search_tool = ToolDefinition(
    name="web_search",
    description="Search the web.",
    parameters={"type": "object", "properties": {"query": {"type": "string"}}},
)

agent = (
    AgentBuilder()
    .with_name("Research Agent")
    .with_description("Searches the web and summarises findings.")
    .with_version("2.1.0")
    .with_tags("prod", "research")
    .with_system_prompt("You are a thorough research assistant.")
    .with_tools([search_tool])
    .with_defaults(max_iterations=8, temperature=0.3)
    .use_bridge(BridgeAgentFactory.for_openai(client, strategy="react"))
    .build()
)
```

Middleware APIs

Middleware is the SDK's cross-cutting interception layer. It combines shared mutable context objects with ordered `next(...)` delegation across runs, streams, model calls, and tool calls.

class BaseAgentMiddleware MIDDLEWARE BASE

Convenience base class with pass-through defaults for all middleware hooks. Override only the hooks you need.

```
async def wrap_run(context: AgentRunContext, handler) → RunResult
```

Wrap a full `Agent.run(...)` execution. Applies to Bridge, Adapter, Native, and workflow-backed runtimes.

```
async def wrap_stream(context: StreamContext, handler) →  
AsyncGenerator[StreamChunk, None]
```

Wrap a full `Agent.stream(...)` execution. Middleware may short-circuit by returning its own async generator.

```
async def wrap_model_call(context: ModelCallContext, handler) → ModelResponse
```

Wrap one Bridge executor model invocation. Available only when Kneo owns the inner loop through Bridge runtimes.

```
async def wrap_tool_call(context: ToolCallContext, handler) → ToolResult
```

Wrap one Bridge executor tool dispatch. Useful for logging, guardrails, retries, or result rewriting.

Context Dataclasses MIDDLEWARE

TYPE	KEY FIELDS	DESCRIPTION
AgentRunContext	<code>agent_name</code> , <code>runtime_name</code> , <code>user_message</code> , <code>messages</code> , <code>run_config</code> , <code>metadata</code>	Shared mutable state for one <code>run()</code> call.
StreamContext	<code>agent_name</code> , <code>runtime_name</code> , <code>user_message</code> , <code>messages</code> , <code>run_config</code> , <code>metadata</code>	Shared mutable state for one <code>stream()</code> call.
ModelCallContext	<code>executor_name</code> , <code>runtime_name</code> , <code>iteration</code> , <code>messages</code> , <code>run_config</code> , <code>metadata</code>	State for one Bridge model call.
ToolCallContext	<code>executor_name</code> , <code>runtime_name</code> , <code>iteration</code> ,	State for one Bridge tool invocation.

messages , run_config ,
tool_call , metadata

ModelResponse	text , tool_calls	Normalized Bridge model-call result returned from wrap_model_call(...) .
---------------	-------------------	--

Ordering: Static middleware from AgentBuilder is applied first. Per-run middleware from RunConfig(middlewares=[...]) is appended after it, so it executes deeper in the chain.

Handler type aliases TYPES

The handler argument each wrap_* hook receives — call it to delegate to the next middleware (or the runtime). Exported from kneo_agent.middleware for typing custom middleware.

ALIAS	SIGNATURE
AgentRunHandler	Callable[[AgentRunContext], Awaitable[RunResult]]
StreamHandler	Callable[[StreamContext], Awaitable[AsyncGenerator[StreamChunk, None]]]
ModelCallHandler	Callable[[ModelCallContext], Awaitable[ModelResponse]]
ToolCallHandler	Callable[[ToolCallContext], Awaitable[ToolResult]]

Production middleware bundle MIDDLEWARE

Ready-to-use middleware implementations, all exported from kneo_agent.middleware and subclassing BaseAgentMiddleware . The model/tool-call hooks apply only on Bridge runtimes (where Kneo owns the inner loop); the run hook applies everywhere.

RetryMiddleware(*, max_attempts=3, initial_delay=0.5, backoff_factor=2.0, max_delay=30.0, jitter=0.1, retry_on=(Exception,), retry_model_calls=True, retry_tool_calls=False)

Retries failed Bridge model calls and/or tool calls with exponential backoff + jitter. `retry_on` selects which exception types are retried.

```
RateLimitMiddleware(*, rate: float, burst: float | None = None, scope: str = "run")
```

Token-bucket rate limiter. `rate` is permits per second; `burst` is the bucket capacity (defaults to `max(rate, 1)`). `scope` is "run" or "model_call".

```
TokenBudgetMiddleware(*, per_run: int | None = None, cumulative: int | None = None, on_missing: str = "ignore")
```

Enforces token budgets from `RunResult.metadata["usage"]`. At least one of `per_run` / `cumulative` is required. `on_missing` ("ignore" / "warn" / "raise") controls behaviour when a run reports no usage. Exposes `cumulative_used` and `reset()` for billing periods. Raises `TokenBudgetExceeded` when a budget is crossed.

```
RedactionMiddleware(patterns: Iterable[str], *, replacement="[REDACTED]",
redact_input=True, redact_output=True, redact_tool_args=True,
redact_tool_results=True)
```

Regex-redacts sensitive strings on the input, output, tool arguments, and/or tool results. Requires at least one pattern — pass `COMMON_PATTERNS` to start.

SYMBOL	KIND	DESCRIPTION
<code>TokenBudgetExceeded</code>	<code>RuntimeError</code>	Raised by <code>TokenBudgetMiddleware</code> when the per-run or cumulative budget is exceeded; carries <code>used</code> , <code>limit</code> , and <code>scope</code> .
<code>COMMON_PATTERNS</code>	<code>tuple[str, ...]</code>	A starter set of redaction regexes (common secret/PII shapes) for <code>RedactionMiddleware</code> ; combine with your own.

Observability

The `kneo_agent.observability` subpackage provides first-party OpenTelemetry support. It is gated behind the optional `[telemetry]` extra (`pip install "kneo-agent[telemetry]"`) so the core package keeps no telemetry dependencies.

- Why this is a separate subpackage, not part of `kneo_agent.utils` :**
- 1. **Optional dependency boundary.** Every name re-exported from `kneo_agent.utils` is import-safe with just the core install. Hosting OTel-backed code there would either force `opentelemetry-api` into the core install or require conditional `try: import opentelemetry` guards in `utils/__init__.py`. A dedicated subpackage makes the `[telemetry]` boundary visually obvious — `from kneo_agent.observability import ...` clearly signals "this requires the extra".
 - 2. **It is a middleware, not a helper.** `OpenTelemetryMiddleware` subclasses `BaseAgentMiddleware` and participates in `run / stream / model-call / tool-call` dispatch. Architecturally it belongs alongside `core/middleware.py`, not in the utility bin where `ToolRegistry` and message constructors live.
 - 3. **Room to grow.** Observability tends to accrete: OTel metrics, log correlation, custom exporters, structured-logging adapters. Starting as its own subpackage gives it space without bloating `utils`. `configure_logging` and `get_logger` remain in `kneo_agent.utils` because they are thin `stdlib` wrappers with no optional deps and don't participate in the middleware contract. If logging ever grows OTel-aware features (log/trace correlation, OTLP log export), the cleaner move is to *also* extract logging here — possibly renaming this subpackage to `instrumentation` — rather than merge `observability` back into `utils`.

class OpenTelemetryMiddleware MIDDLEWARE

A `BaseAgentMiddleware` subclass that emits spans for all four hook points (`run`, `stream`, `model call`, `tool call`) following the [OpenTelemetry GenAI semantic conventions](#).

```
def __init__(tracer: Tracer | None = None, *, record_arguments: bool = True,
record_results: bool = False)
```

PARAMETER	TYPE	DESCRIPTION
tracer	<code>opentelemetry.trace.Tracer</code> <code>None</code>	Pre-configured tracer. If <code>None</code> , a tracer named "kneo_agent" is acquired from the global tracer provider.

<code>record_arguments</code>	<code>bool</code>	Serialize tool-call arguments into <code>gen_ai.tool.call.arguments</code> . Disable when arguments may contain PII. Default: <code>True</code> .
<code>record_results</code>	<code>bool</code>	Attach tool results to spans. Off by default since results can be large or sensitive.

Span hierarchy

<code>chat {agent_name}</code>	<code>(wrap_run)</code>
└─ <code>chat iter=1</code>	<code>(wrap_model_call)</code>
└─ <code>execute_tool {tool}</code>	<code>(wrap_tool_call)</code>
└─ <code>chat iter=2</code>	<code>(wrap_model_call)</code>

Span attributes

ATTRIBUTE	WHEN EMITTED	SOURCE
<code>gen_ai.system</code>	all spans	<code>runtime.name</code>
<code>gen_ai.operation.name</code>	all spans	"chat" for runs and model calls; "execute_tool" for tool spans
<code>gen_ai.agent.name</code>	run + stream spans	<code>Agent.name</code>
<code>gen_ai.tool.name</code>	tool spans	<code>tool_call.name</code>
<code>gen_ai.tool.call.id</code>	tool spans	<code>tool_call.id</code>
<code>gen_ai.tool.call.arguments</code>	tool spans (when <code>record_arguments=True</code>)	JSON-serialised <code>tool_call.arguments</code> , truncated at 4096 chars
<code>gen_ai.request.model</code>	any span	<code>RunConfig.extra["model"]</code> when set
<code>gen_ai.request.temperature</code>	any span	<code>RunConfig.temperature</code>

<code>gen_ai.usage.input_tokens</code>	run spans	<code>RunResult.metadata["usage"]</code> ["input_tokens"] or "prompt_tokens" when surfaced by the runtime
<code>gen_ai.usage.output_tokens</code>	run spans	<code>RunResult.metadata["usage"]</code> ["output_tokens"] or "completion_tokens" when surfaced by the runtime

Exceptions raised by downstream handlers are recorded on the active span via `record_exception` and the span status is set to `ERROR` before re-raising.

Install: `pip install "kneo-agent[telemetry]"`. Importing the module without the extra installed raises `ImportError` at `OpenTelemetryMiddleware()` construction.

Workflows

Composite workflows provide a workflow-as-agent model. Each participant receives the full conversation history, appends its own contribution, and passes the updated message list to the next participant.

The workflow package is split by runtime type: `workflow.py` contains the graph runtime, `sequential_workflow.py` contains the default orchestration runtime, `concurrent_workflow.py`, `handoff_workflow.py`, and `group_chat_workflow.py` contain the specialized orchestration runtimes, and `builders.py` contains the workflow and orchestration builders.

Workflow GRAPH RUNTIME

General graph workflow with explicit executors and edges. Execution proceeds in supersteps: run the current executor set, merge results into shared history, then route along matching edges. Like the other workflow runtimes, it also satisfies `AgentRuntime` and can be wrapped as a regular agent.

CONSTRUCTOR PARAMETER	TYPE	DESCRIPTION
<code>executors</code>	<code>dict[str, WorkflowComponent]</code>	Registered workflow executors by name.

<code>start_executor_ids</code>	<code>list[str]</code>	Executor names scheduled in the first superstep.
<code>edges</code>	<code>list[WorkflowEdge]</code>	Directed routing edges between executors.

```
def as_agent(*, name: str | None = None, description: str = "",
             system_prompt: str | None = None) → Agent
```

Wrap the graph workflow in the regular `Agent` facade so callers can use `run()`, `stream()`, and `chat()` through the existing architecture.

WorkflowEdge DATACLASS

Represents a directed edge between two workflow executors.

FIELD	TYPE	DESCRIPTION
<code>source</code>	<code>str</code>	Source executor name.
<code>target</code>	<code>str</code>	Target executor name.
<code>condition</code>	<code>Callable None</code>	Optional route predicate evaluated after the source executor runs.
<code>label</code>	<code>str</code>	Optional human-readable edge label.

WorkflowComponent PROTOCOL

Protocol implemented by workflow participants. A participant must expose `name`, support `run(messages, config)`, and report whether it supports streaming.

MEMBER	TYPE	DESCRIPTION
<code>name</code>	<code>str</code>	Human-readable participant identifier.
<code>run(messages, config)</code>	<code>RunResult</code>	Execute the participant against the full conversation history.

<code>supports_streaming()</code>	<code>bool</code>	Whether the participant can stream.
-----------------------------------	-------------------	-------------------------------------

Built-In Orchestration Runtimes FAMILY

Kneo Agent ships four participant-oriented orchestration runtimes with the same overall shape. Internally they share the same `OrchestrationBase`, and all of them satisfy both `WorkflowComponent` and `AgentRuntime`, can be nested, and can be wrapped as regular agents with `as_agent(...)`.

RUNTIME	PURPOSE	DEFAULT SHAPE
<code>SequentialWorkflow</code>	Default orchestration runtime	Runs participants in order against the evolving shared conversation.
<code>ConcurrentWorkflow</code>	Fan-out orchestration	Runs all participants against the same input snapshot in parallel.
<code>HandoffWorkflow</code>	Selector-driven orchestration	Chooses the next participant dynamically at runtime.
<code>GroupChatWorkflow</code>	Round-robin orchestration	Cycles through participants in a shared conversation for a configured number of rounds.

SequentialWorkflow ORCHESTRATION

The default orchestration runtime. It runs participants in order and shares the same workflow/runtime surface as the other built-in orchestration runtimes.

CONSTRUCTOR PARAMETER	TYPE	DESCRIPTION
<code>participants</code>	<code>list[Agent AgentRuntime WorkflowComponent FunctionStep]</code>	Ordered workflow participants. The list must not be empty.
<code>name</code>	<code>str</code>	Workflow runtime name. Exposed as <code>runtime_name</code> when wrapped as an agent.
<code>description</code>	<code>str</code>	Human-readable description for docs or registries.

```
async def run(messages: list[Message], config: RunConfig) → RunResult
```

Execute all workflow participants in order. The final result aggregates the full conversation, tool results, total iterations, and workflow metadata such as `workflow_steps` .

```
def as_agent(*, name: str | None = None, description: str = "",
system_prompt: str | None = None) → Agent
```

Wrap the workflow in the regular `Agent` facade so callers can use `run()` , `stream()` , and `chat()` exactly as they would for any other agent.

ConcurrentWorkflow ORCHESTRATION

Fan-out orchestration. Runs all participants against the same input snapshot in parallel and merges their output back into shared history.

Uses the same participant-oriented runtime surface as `SequentialWorkflow` , but changes the execution policy from ordered execution to parallel fan-out.

HandoffWorkflow ORCHESTRATION

Dynamic specialist routing. A selector callback chooses which participant handles the next turn.

Uses the same participant-oriented runtime surface as `SequentialWorkflow` , but changes the execution policy to selector-driven routing.

GroupChatWorkflow ORCHESTRATION

Round-robin shared conversation. Participants speak in order for a configured number of rounds.

Uses the same participant-oriented runtime surface as `SequentialWorkflow` , but changes the execution policy to round-robin conversation turns.

FunctionStep WORKFLOW LEAF

Custom workflow participant backed by a Python callable. The handler receives the full conversation history and current `RunConfig` .

FIELD	TYPE	DESCRIPTION
<code>step_name</code>	<code>str</code>	Workflow step name.

<code>handler</code>	<code>Callable[[list[Message], RunConfig], WorkflowReturn]</code>	Function or coroutine used to generate the step output.
<code>description</code>	<code>str</code>	Optional human-readable description.

A handler may return `RunResult`, `Message`, `list[Message]`, `str`, or `None`.

`FunctionExecutor` is an exported alias for `FunctionStep` so workflow leaves can be named as executors when that terminology is clearer.

HumanInTheLoopStep WORKFLOW LEAF

Workflow participant that requests explicit human review or approval. It can resolve inline through a callback or pause execution by raising `HumanInterventionRequiredError`.

FIELD	TYPE	DESCRIPTION
<code>step_name</code>	<code>str</code>	Workflow step name.
<code>prompt</code>	<code>str</code> <code>callable</code>	Prompt shown to the human reviewer. May be computed from the current messages and run config.
<code>resolver</code>	<code>callable</code> <code>None</code>	Optional callback that returns the human response inline.
<code>description</code>	<code>str</code>	Human-readable step description. Default: <code>" "</code> .
<code>response_role</code>	<code>str</code>	Message role used when the human response is appended. Default: <code>"user"</code> .
<code>metadata</code>	<code>dict</code>	Arbitrary key/value metadata carried on the step and surfaced in pause payloads. Default: <code>{}</code> .

HumanInterventionRequest DATA CLASS

Structured request object passed to human-step resolvers and surfaced in pause exceptions.

RetryStep WORKFLOW LEAF

Wraps any `WorkflowComponent` with retry-on-failure. Parameters mirror `RetryMiddleware` minus the per-hook flags (a workflow step has a single `run` entry point).

FIELD	TYPE	DEFAULT	DESCRIPTION
<code>component</code>	<code>WorkflowComponent</code>	—	The step to wrap.
<code>max_attempts</code>	<code>int</code>	3	Total attempts before giving up.
<code>initial_delay</code>	<code>float</code>	0.5	Delay before the first retry (seconds).
<code>backoff_factor</code>	<code>float</code>	2.0	Multiplier applied to the delay each retry.
<code>max_delay</code>	<code>float</code>	30.0	Upper bound on any single backoff delay.
<code>jitter</code>	<code>float</code>	0.1	Random jitter fraction added to each delay.
<code>retry_on</code>	<code>tuple[type[BaseException], ...]</code>	<code>(Exception,)</code>	Exception types that trigger a retry.

WorkflowBuilder FACTORY

Builder for graph workflows plus convenience helpers for workflow leaves and common orchestration patterns.

METHOD	RETURN	DESCRIPTION
<code>WorkflowBuilder(start_executor, *, name="workflow", description="")</code>	<code>WorkflowBuilder</code>	Create a graph workflow builder with an initial executor.
<code>.add_executor(executor)</code>	<code>WorkflowBuilder</code>	Register another executor in the graph.
<code>.add_edge(source, target, *, condition=None, label="")</code>	<code>WorkflowBuilder</code>	Add a routing edge between executors.

<code>.build()</code>	Workflow	Build the graph workflow.
<code>.step(name, handler, *, description="")</code>	FunctionStep	Create a custom function-based workflow participant.
<code>.human_step(name, prompt, *, resolver=None, description="", response_role="user", metadata=None)</code>	HumanInTheLoopStep	Create a workflow participant that requests explicit human input.
<code>.sequential(participants, ...)</code>	SequentialWorkflow	Convenience helper for the default linear orchestration runtime.
<code>.concurrent(participants, ...)</code>	ConcurrentWorkflow	Convenience helper for fan-out composition.

Orchestration Builders

FACTORIES

BUILDER	BUILDS	DESCRIPTION
<code>SequentialBuilder</code>	SequentialWorkflow	Default linear orchestration runtime.
<code>ConcurrentBuilder</code>	ConcurrentWorkflow	Parallel fan-out orchestration.
<code>HandoffBuilder</code>	HandoffWorkflow	Selector-driven specialist routing.
<code>GroupChatBuilder</code>	GroupChatWorkflow	Round-robin shared conversation orchestration.

```
from kneo_agent import AgentBuilder
from kneo_agent.workflows import WorkflowBuilder

writer = AgentBuilder().with_name("writer").use_runtime(writer_runtime).build()
reviewer =
AgentBuilder().with_name("reviewer").use_runtime(reviewer_runtime).build()

editorial = WorkflowBuilder.sequential([writer, reviewer], name="editorial")
publish = WorkflowBuilder.step("publish", lambda messages, config: "Published.")

release = WorkflowBuilder.sequential([editorial, publish], name="release")
workflow_agent = release.as_agent(name="Release Workflow")
```

Core Data Classes

AgentConfig DATACLASS

Static metadata that describes *what the agent is*. Constructed by `AgentBuilder.build()`. Exposed via `agent.config`.

FIELD	TYPE	DEFAULT	DESCRIPTION
name	str	—	Agent display name.
description	str	""	Human-readable description.
version	str	"1.0.0"	Semantic version.
system_prompt	str None	None	Default system prompt for all runs.
tools	list[ToolDefinition]	[]	Tool definitions passed to every run.
middlewares	list[AgentMiddleware]	[]	Static middleware chain attached to the agent.

<code>default_run_config</code>	<code>dict[str, Any]</code>	<code>{}</code>	RunConfig field overrides applied to every run.
<code>tags</code>	<code>list[str]</code>	<code>[]</code>	Arbitrary labels for grouping/filtering.

RunConfig DATACLASS

Per-run configuration describing *how a particular run behaves*. Merged:
AgentConfig.default_run_config → run_config kwarg → **extra kwargs.

FIELD	TYPE	DEFAULT	DESCRIPTION
<code>max_iterations</code>	<code>int</code>	<code>10</code>	Maximum agentic loop iterations before stopping.
<code>temperature</code>	<code>float</code>	<code>0.7</code>	Sampling temperature forwarded to the model.
<code>system_prompt</code>	<code>str None</code>	<code>None</code>	Overrides AgentConfig.system_prompt for this run.
<code>tools</code>	<code>list[ToolDefinition]</code>	<code>[]</code>	Overrides the agent's tool list for this run.
<code>middlewares</code>	<code>list[AgentMiddleware]</code>	<code>[]</code>	Additional middleware appended for this single invocation.
<code>extra</code>	<code>dict[str, Any]</code>	<code>{}</code>	Provider-specific key/value pairs forwarded verbatim (e.g. tool_handlers for OpenAI provider).

```
# Override temperature and add a per-run middleware/tool handler
result = await agent.run(
    "Summarise the latest AI news.",
    run_config=RunConfig(
        temperature=0.2,
        max_iterations=5,
        middlewares=[MyLoggingMiddleware()],
        extra={"tool_handlers": {"web_search": my_search_fn}},
    ),
)
```

RunResult DATACLASS

Everything produced by a completed `agent.run()` call.

FIELD	TYPE	DESCRIPTION
<code>final_message</code>	<code>str</code>	The agent's final text response.
<code>messages</code>	<code>list[Message]</code>	Full conversation as it stood at run end, including tool messages.
<code>iterations</code>	<code>int</code>	Number of LLM calls made in the loop.
<code>tool_calls_performed</code>	<code>list[ToolResult]</code>	All tool calls executed during the run.
<code>duration_ms</code>	<code>float</code>	Wall-clock time in milliseconds.
<code>metadata</code>	<code>dict[str, Any]</code>	Strategy-specific extras, e.g. <code>{"plan": "..."} from Plan-Act, <code>{"stopped_by_max_iterations": True}</code>.</code>

```
result = await agent.run("Research quantum computing.")

print(result.final_message)
print(f"Iterations: {result.iterations}")
print(f"Duration: {result.duration_ms:.1f} ms")
print(f"Tools used: {[r.name for r in result.tool_calls_performed]}")

if result.metadata.get("plan"):
    print(f"Plan used:\n{result.metadata['plan']}")

if result.metadata.get("stopped_by_max_iterations"):
    print("Warning: loop was cut short by max_iterations.")
```

StreamChunk DATACLASS

A single unit yielded by a streaming agent run. Always check `chunk.type` before accessing optional fields.

FIELD	TYPE	DESCRIPTION
<code>type</code>	<code>"text" "tool_call" "tool_result" "done"</code>	Discriminator. Always present.
<code>content</code>	<code>str None</code>	Text token. Present only when <code>type == "text"</code> .
<code>tool_call</code>	<code>ToolCall None</code>	Tool invocation details. Present only when <code>type == "tool_call"</code> .
<code>tool_result</code>	<code>ToolResult None</code>	Tool execution result. Present only when <code>type == "tool_result"</code> .

Message DATACLASS

A single turn in an agent conversation.

FIELD	TYPE	DESCRIPTION
-------	------	-------------

role	"user" "assistant" "system" "tool"	Message role.
content	str	Message text.
name	str None	Tool name. Populated for role="tool" messages.
tool_call_id	str None	Links a tool result to its originating call. Populated for role="tool" messages.

ToolDefinition DATACLASS

FIELD	TYPE	DESCRIPTION
name	str	Tool name used by the model to call it.
description	str	Natural-language description shown to the model.
parameters	dict[str, Any]	JSON Schema object describing arguments.

ToolCall DATACLASS

FIELD	TYPE	DESCRIPTION
id	str	Unique call identifier (from the model).
name	str	Tool name to invoke.
arguments	dict[str, Any]	Parsed JSON arguments from the model.

ToolResult DATACLASS

FIELD	TYPE	DESCRIPTION
-------	------	-------------

<code>tool_call_id</code>	<code>str</code>	Links back to the originating <code>ToolCall.id</code> .
<code>name</code>	<code>str</code>	Tool name.
<code>result</code>	<code>str</code>	Serialised result string returned to the model.

AgentRuntime PROTOCOL

The structural protocol that every runtime must satisfy. `AgentExecutor` subclasses (Bridge), platform-owned native runtimes, and `*Adapter` classes all implement this. The `Agent` class only ever depends on this protocol.

MEMBER	SIGNATURE	DESCRIPTION
<code>name</code>	<code>str (property)</code>	Human-readable identifier (e.g. "react@openai-agents", "react@langchain", or "google-adk-native").
<code>run()</code>	<code>async → RunResult</code>	Execute a full agent run.
<code>stream()</code>	<code>def → AsyncGenerator[StreamChunk, None]</code>	Async generator yielding run chunks. Declared as <code>def</code> (not <code>async def</code>) so implementations can be async generator functions and call sites iterate with <code>async for chunk in runtime.stream(...)</code> directly.
<code>supports_streaming()</code>	<code>bool</code>	Whether <code>stream()</code> is supported.
<code>supports_tools()</code>	<code>bool</code>	Whether the runtime dispatches tool calls.

Role TYPE ALIAS

`kneo_agent.Role` — `Literal["user", "assistant", "system", "tool"]`. The set of allowed values for `Message.role`.

BridgeAgentFactory

class BridgeAgentFactory FACTORY

Static factory that constructs Kneo-owned Bridge runtimes. Strategy and platform are chosen independently when the underlying provider satisfies the Bridge `RuntimeImpl` contract.

Scope: LangChain Bridge remains a true Bridge. Google ADK Bridge is a compatibility translation layer. OpenAI is kept as a compatibility alias and now returns a native runtime with a warning.

```
@staticmethod for_google_adk(adk_runner, session_id: str, strategy: BridgeStrategy = "react") → AgentRuntime
```

PARAMETER	TYPE	DESCRIPTION
adk_runner	Any	A <code>google.adk.runners.InMemoryRunner</code> or compatible instance.
session_id	str	ADK session identifier for this conversation.
strategy	BridgeStrategy	"simple" , "react" , or "plan-act" . Default: "react" .

Warning: this is a compatibility bridge over ADK-shaped payloads. For native Google ADK loop ownership, prefer `NativeRuntimeFactory.for_google_adk(...)` or `AdapterAgentFactory.for_google_adk(...)` .

```
@staticmethod for_openai(openai_client = None, model: str = "gpt-4o", strategy: BridgeStrategy = "react", runner = None, *, base_url: str | None = None, api_key: str | None = None) → AgentRuntime
```

PARAMETER	TYPE	DESCRIPTION
openai_client	Any None	Optional <code>AsyncOpenAI</code> client used by the OpenAI Agents SDK

chat-completions model wrapper.

model	str	Model identifier. Default: "gpt-4o" .
strategy	BridgeStrategy	Strategy label preserved in the runtime name.
runner	Any None	Optional OpenAI Agents SDK runner override, mainly useful for tests.

Compatibility alias. This now returns the platform-owned OpenAI Agents SDK runtime and emits a warning. Prefer `NativeRuntimeFactory.for_openai(...)` .

```
@staticmethod for_langchain(chat_model, tool_registry: dict | None = None,
                             strategy: BridgeStrategy = "react") → AgentRuntime
```

PARAMETER	TYPE	DESCRIPTION
chat_model	Any	Any <code>langchain_core.language_models.BaseChatModel</code> subclass.
tool_registry	dict[str, Callable] None	Tool name → callable mapping. Can also be registered post-construction via <code>LangChainImpl.register_tools()</code> .
strategy	BridgeStrategy	Loop strategy. Default: "react" .

```
@staticmethod custom(impl: RuntimeImpl, strategy: BridgeStrategy = "react") → AgentRuntime
```

Bridge a custom `RuntimeImpl` with any strategy. Use this to plug in a provider not natively supported by Kneo Agent.

```

from kneo_agent.patterns import BridgeAgentFactory, NativeRuntimeFactory
from langchain_openai import ChatOpenAI

# True Bridge: Kneo loop over a LangChain chat model
lc_react_rt = BridgeAgentFactory.for_langchain(ChatOpenAI(), strategy="react")

# Native runtime: platform owns the loop
oai_native_rt = NativeRuntimeFactory.for_openai(model="gpt-4o", strategy="react")

print(oai_native_rt.name) # "react@openai-agents"
print(lc_react_rt.name) # "react@langchain"

```

NativeRuntimeFactory

class NativeRuntimeFactory FACTORY

Constructs platform-owned runtimes. Use this when the underlying SDK/runtime should own the loop instead of Kneo.

```

@staticmethod for_openai(openai_client = None, model: str = "gpt-4o",
strategy: BridgeStrategy = "react", runner = None, *, base_url: str | None =
None, api_key: str | None = None) → OpenAIAgentsRuntime

```

Returns the OpenAI Agents SDK-backed runtime. The SDK owns the loop; the strategy is preserved in the runtime name and for Plan-Act behavior.

```

@staticmethod for_google_adk(adk_runner, app_name: str, user_id: str,
session_id: str) → GoogleADKRuntime

```

Returns the native Google ADK runtime backed by `runner.run_async(...)`. Google ADK owns the loop.

```
from kneo_agent.patterns import NativeRuntimeFactory

oai_runtime = NativeRuntimeFactory.for_openai(model="gpt-4o")
adk_runtime = NativeRuntimeFactory.for_google_adk(runner, "app", "user-1",
"session-1")
```

AdapterAgentFactory

class AdapterAgentFactory FACTORY

Constructs Adapter-pattern runtimes that wrap existing platform objects. The platform manages its own loop; no strategy choice is needed or possible.

```
@staticmethod for_google_adk(adk_runner, app_name: str, user_id: str,
session_id: str) → GoogleADKAdapter
```

Wraps an ADK runner that exposes `run_async(app_name, user_id, session_id, new_message)` as an async generator.

```
@staticmethod for_openai(runner, agent_definition: dict) → OpenAIAdaptersAdapter
```

Wraps an `@openai/agents Runner`. `agent_definition` requires at minimum `{"name": ..., "instructions": ...}`.

```
@staticmethod for_langchain(agent_executor) → LangChainAdapter
```

Wraps a `LangChain AgentExecutor` (or `CompiledGraph`) with `ainvoke` and optionally `astream`.

```

from kneo_agent.patterns import AdapterAgentFactory

# Wrap an already-running LangChain AgentExecutor
lc_executor = build_langchain_agent() # your existing code, untouched
runtime = AdapterAgentFactory.for_langchain(lc_executor)

agent = AgentBuilder().use_adapter(runtime).build()
reply = await agent.chat("What can you do?")

```

Bridge Executors

The Bridge abstraction side. These are Kneo-owned loop strategies. Inject a `RuntimeImpl` at construction time. Use Native runtimes when the platform should own the loop.

SimpleAgentExecutor EXECUTOR

One-shot completion. No loop, no tool calls. `name` → `"simple@<platform>"`.

```

from kneo_agent import SimpleAgentExecutor
from kneo_agent.providers import LangChainImpl

impl = LangChainImpl(chat_model)
runtime = SimpleAgentExecutor(impl)
# runtime.name == "simple@langchain"

```

ReActAgentExecutor EXECUTOR

Reason → Act → Observe loop. Tool calls are executed concurrently with `asyncio.gather`. Terminates when the model returns no tool calls or `max_iterations` is reached. `name` → `"react@<platform>"`.

Loop invariants

- All tool calls within a single model response are dispatched in parallel.
- Tool results are appended as `role="tool"` messages before the next completion.
- If `max_iterations` is reached, `RunResult.metadata["stopped_by_max_iterations"] == True`.

```

from kneo_agent import ReActAgentExecutor
from kneo_agent.providers import LangChainImpl

impl = LangChainImpl(chat_model)
runtime = ReActAgentExecutor(impl)
agent = AgentBuilder().with_tools([search_tool]).use_bridge(runtime).build()

result = await agent.run("What is the latest news on AI?")
if result.metadata.get("stopped_by_max_iterations"):
    print("Warning: loop hit max_iterations")

```

PlanActAgentExecutor EXECUTOR

Two-phase strategy: generate a numbered plan (tools disabled), then execute it with the full ReAct loop. The plan is stored in `RunResult.metadata["plan"] . name → "plan-act@<platform>"`.

When to use Plan-Act

- Complex tasks with 3+ sequential dependent steps.
- Tasks where implicit ReAct loops tend to lose track of the goal.
- When you want a human-readable trace of intent before execution.

```

from kneo_agent import PlanActAgentExecutor
from kneo_agent.providers import LangChainImpl

runtime = PlanActAgentExecutor(LangChainImpl(chat_model))
agent = AgentBuilder().with_tools(tools).use_bridge(runtime).build()

result = await agent.run("Research and write a report on quantum computing.")
print("Plan:\n", result.metadata["plan"])
print("\nReport:\n", result.final_message)

```

Platform Adapters

The Adapter pattern side. Each class wraps a fixed-interface platform object and implements `AgentRuntime`. The platform owns the agentic loop.

GoogleADKAdapter ADAPTER

Wraps an ADK runner whose `run_async()` method yields typed events. ADK handles tool dispatch, retries, and multi-turn internally. `name` → "google-adk-adapter".

```
from google.adk.runners import InMemoryRunner
from kneo_agent.runtime.adapters import GoogleADKAdapter

runner = InMemoryRunner(agent=my_adk_agent, app_name="my-app")
adapter = GoogleADKAdapter(runner, app_name="my-app", user_id="u-1",
session_id="s-1")
agent = AgentBuilder().use_adapter(adapter).build()
```

OpenAIAgentsAdapter ADAPTER

Wraps an `@openai/agents Runner`. Translates the SDK's `list[Message]` into the runner's single `input: str` and maps `new_items` back to `ToolResult[]`. `name` → "openai-agents-adapter".

```
from agents import Agent, Runner    # @openai/agents
from kneo_agent.runtime.adapters import OpenAIAgentsAdapter

oai_agent = Agent(name="helper", instructions="Be helpful.", tools=[...])
runner = Runner()
adapter = OpenAIAgentsAdapter(
    runner,
    agent_definition={"name": "helper", "instructions": "Be helpful."},
)
agent = AgentBuilder().use_adapter(adapter).build()
```

LangChainAdapter ADAPTER

Wraps a LangChain `AgentExecutor`. Translates `list[Message]` → (input, chat_history) tuple format. `intermediate_steps` becomes `ToolResult[]`. `name` → "langchain-adapter".

Streaming: Only available if the executor exposes `astream()` .

```
from langchain.agents import AgentExecutor, create_openai_functions_agent
from kneo_agent.runtime.adapters import LangChainAdapter

lc_executor = AgentExecutor(agent=..., tools=[])
adapter = LangChainAdapter(lc_executor)
kneo_agent = AgentBuilder().use_adapter(adapter).build()

# Multi-turn: the adapter correctly extracts chat_history from history
await kneo_agent.chat("Hello!")
await kneo_agent.chat("What did I just say?")  # history is carried forward
```

RuntimeImpl Protocol

RuntimeImpl PROTOCOL

The Implementor interface in the Bridge pattern. Concrete Bridge providers such as `GoogleADKImpl` (compatibility layer) and `LangChainImpl` implement this. Native runtimes such as `OpenAIAgentsRuntime` do not implement `RuntimeImpl` because the platform owns the loop directly.

METHOD	SIGNATURE	DESCRIPTION
<code>platform_name</code>	<code>str</code> (property)	Short platform ID (e.g. "langchain" or "google-adk").
<code>complete()</code>	<code>async → (str, list[ToolCall])</code>	Send one completion request; return (text, tool_calls) .
<code>execute_tool()</code>	<code>async → str</code>	Dispatch a tool call and return its result.
<code>stream_tokens()</code>	<code>async generator → str</code>	Yield raw text tokens.
<code>supports_tools()</code>	<code>bool</code>	Whether tool calling is supported.

<code>supports_streaming()</code>	<code>bool</code>	Whether <code>stream_tokens()</code> is supported.
-----------------------------------	-------------------	--

Providers

GoogleADKImpl PROVIDER

`kneo_agent.providers.google_adk` — Compatibility `RuntimeImpl` for Google ADK. Translates `list[Message]` / `RunConfig` into ADK-shaped payloads so Kneo Bridge executors can drive the loop.

Install: `pip install kneo-agent[google-adk]`

Important: This is not the native Google ADK loop-owned runtime. For that, use `NativeRuntimeFactory.for_google_adk(...)` or `GoogleADKRuntime`.

GoogleADKImpl(`adk_runner: Any`, `session_id: str`)

```
from google.adk.runners import InMemoryRunner
from kneo_agent.providers import GoogleADKImpl
from kneo_agent import ReActAgentExecutor

runner = InMemoryRunner(agent=my_adk_agent, app_name="app")
impl = GoogleADKImpl(runner, session_id="session-1")
runtime = ReActAgentExecutor(impl)      # or SimpleAgentExecutor,
PlanActAgentExecutor
```

OpenAIAgentsImpl NATIVE RUNTIME

`kneo_agent.providers.openai_agents` — OpenAI Agents SDK-backed native runtime. Uses the SDK's `Agent`, `Runner.run()`, and `Runner.run_streamed()` APIs directly.

Install: `pip install kneo-agent[openai]`

OpenAIAgentsImpl(`openai_client: Any | None = None`, `model: str = "gpt-4o"`,
`strategy: str = "react"`, `runner: Any | None = None`)

Tool execution: Pass tool handlers in `RunConfig.extra["tool_handlers"]` as a `dict[str, Callable]`. They are converted into OpenAI Agents SDK `FunctionTool` instances.

```
from kneo_agent.patterns import NativeRuntimeFactory
from kneo_agent import RunConfig

runtime = NativeRuntimeFactory.for_openai(model="gpt-4o", strategy="react")

result = await agent.run(
    "Search for Python tutorials.",
    run_config=RunConfig(extra={"tool_handlers": {"web_search": my_search_fn}}),
)
```

LangChainImpl PROVIDER

`kneo_agent.providers.langchain` — Wraps any `BaseChatModel` subclass. Tool handlers are registered in a plain Python dict — no `StructuredTool` wrapping required.

Install: `pip install kneo-agent[langchain]`

```
LangChainImpl(chat_model: Any, tool_registry: dict | None = None)
```

```
def register_tool(name: str, handler: Callable) → None
```

```
from langchain_openai import ChatOpenAI
from kneo_agent.providers import LangChainImpl
from kneo_agent import ReActAgentExecutor

impl = LangChainImpl(ChatOpenAI(model="gpt-4o"))
impl.register_tool("get_weather", lambda args: f"22 °C in {args['city']}")

runtime = ReActAgentExecutor(impl)
```

Skill APIs

Skills are reusable capability bundles that compile into `RunConfig` . Kneo Agent supports in-memory skills, Agent Skills-compatible `SKILL.md` directories, skill discovery, and on-demand loading of progressive-disclosure resources such as `references/` , `scripts/` , and `assets/` .

SKILL_STANDARD_URL CONSTANT

`kneo_agent.SKILL_STANDARD_URL` — `"https://agentskills.io/specification"` . The Agent Skills specification URL the `SKILL.md` loader/format follows.

Skill DATACLASS

`kneo_agent.core.skills` — reusable skill definition plus loader and resource helpers.

FIELD	TYPE	DESCRIPTION
<code>name</code>	<code>str</code>	Agent Skills-compatible skill name. Must match the parent directory name when loaded from disk.
<code>description</code>	<code>str</code>	Required short description from <code>SKILL.md</code> frontmatter.
<code>system_prompt</code>	<code>str None</code>	Markdown body of the skill. This is the instruction payload compiled into agent runs.
<code>tools</code>	<code>list[ToolDefinition]</code>	Tool definitions declared by the skill.
<code>defaults / extra / tags</code>	<code>dict list</code>	Default run settings, provider extras such as <code>tool_handlers</code> , and arbitrary routing labels.
<code>license / compatibility / metadata / allowed_tools</code>	<code>str dict</code>	Agent Skills frontmatter metadata preserved by the loader.
<code>source_path</code>	<code>str None</code>	Absolute path to the loaded <code>SKILL.md</code> file when sourced

from disk.

```
classmethod from_path(path: str | Path) → Skill
```

Load a skill from either a skill directory or a direct `SKILL.md` file.

```
def with_tool_handlers(handlers: dict[str, Any]) → Skill
```

Return a copy of the skill with `extra["tool_handlers"]` merged in for runtime execution.

```
def resource_paths() → list[str]
```

List bundled skill resources under `references/`, `scripts/`, and `assets/`, relative to the skill root.

```
def read_resource(relative_path: str) → str
```

Read a bundled resource file safely relative to the skill directory. Path escapes are rejected.

```
def activation_prompt() → str
```

Render a structured activation payload containing the skill body and discoverable bundled resources. Useful when integrating with prompt-based skill activation flows.

```
from kneo_agent import load_skill

skill = load_skill("examples/skills/weather")
skill = skill.with_tool_handlers({"get_weather": get_weather})

print(skill.resource_paths())
print(skill.read_resource("references/REFERENCE.md"))
print(skill.activation_prompt())
```

Skill Discovery FUNCTIONS

`discover_skills`, `discover_default_skills`, and `load_skill` provide startup-time discovery and runtime activation for Agent Skills directories.

```
def load_skill(path: str | Path) → Skill
```

Load a skill from a skill directory or direct `SKILL.md` path.

```
def discover_skills(roots: list[str | Path], max_depth: int = 4) → list[SkillCatalogEntry]
```

Scan one or more roots for `SKILL.md` files and return lightweight catalog metadata only.

```
def discover_default_skills(project_root: str | Path | None = None, user_home: str | Path | None = None, max_depth: int = 4) → list[SkillCatalogEntry]
```

Scan conventional project and user locations such as `.agents/skills` and `.claude/skills`.

```
SkillCatalogEntry(name: str, description: str, path: str) → dataclass
```

Lightweight startup-time metadata used for skill menus and registries without loading full resource trees.

Standard: The loader expects an Agent Skills-style `SKILL.md` with YAML frontmatter and a Markdown body. Kneo Agent preserves progressive disclosure by listing bundled files but leaving resource loading explicit and on demand. See [Agent Skills Guide](#) below for the full integration model.

ToolRegistry

class ToolRegistry UTILITY

`kneo_agent.utils.tools` — Maps tool names to `(ToolDefinition, handler)` pairs. Thread-safe for reads. Supports sync and async handlers.

```
def register(definition: ToolDefinition, handler: Callable) → None
```

Explicitly register a tool definition and handler. Overwrites if the name already exists (logs a warning).

```
def register_agent_tool(tool: AgentTool) → None
```

Register an `AgentTool` returned by `Agent.as_tool()`.

```
def add_agent(agent: Agent, **kwargs) → AgentTool
```

Create an `AgentTool` from an existing agent, register it immediately, and return it.

```
@registry.tool(name, description, parameters) → decorator
```

Decorator that registers the decorated function as a tool. Returns the original function unmodified.

```
async def call(tool_call: ToolCall) → str
```

Dispatch a `ToolCall` to its handler. Supports both sync and async handlers. Returns the result coerced to a JSON string.

RAISES

`KeyError` if no handler is registered for `tool_call.name`.

```
def call_sync(tool_call: ToolCall) → str
```

Synchronous variant. Raises `RuntimeError` if the handler is async.

```
async def register_mcp_server(server: MCPServerConfig, *, prefix: str | None = None) → list[ToolDefinition]
```

Connect to an MCP server, discover its tools, and register proxy handlers for them in the registry.

```
async def aclose() → None
```

Close background MCP sessions and subprocesses owned by the registry.

Properties

PROPERTY	TYPE	DESCRIPTION
<code>definitions</code>	<code>list[ToolDefinition]</code>	All registered <code>ToolDefinition</code> objects (pass to <code>AgentBuilder.with_tools()</code>).
<code>names</code>	<code>list[str]</code>	All registered tool names.
<code>len(registry)</code>	<code>int</code>	Number of registered tools.
<code>"name" in registry</code>	<code>bool</code>	Check if a tool is registered.

```
def to_skill(name: str, description: str = "", system_prompt: str | None = None, defaults: dict | None = None, extra: dict | None = None, tags: list[str]
```

| **None** = **None**) → Skill

Package the registry as a `Skill` with both tool definitions and `tool_handlers` merged into `extra`.

```

import json
from kneo_agent.utils import ToolRegistry
from kneo_agent import ToolCall

registry = ToolRegistry()

# Synchronous handler
@registry.tool(
    name="get_weather",
    description="Return current weather for a city.",
    parameters={
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"],
    },
)
def get_weather(args: dict) -> str:
    return json.dumps({"city": args["city"], "temp_c": 22})

# Async handler – both are supported
@registry.tool(
    name="get_time",
    description="Return the current UTC time.",
    parameters={"type": "object", "properties": {}},
)
async def get_time(args: dict) -> str:
    import datetime
    return datetime.datetime.utcnow().isoformat() + "Z"

# Agent-as-tool registration
weather_tool = weather_agent.as_tool(name="get_weather", arg_name="city")
registry.register_agent_tool(weather_tool)

# Use in agent
agent =
AgentBuilder().with_tools(registry.definitions).use_bridge(runtime).build()

```

```
# Dispatch directly (e.g. for testing)
tc = ToolCall(id="1", name="get_weather", arguments={"city": "Tokyo"})
result = await registry.call(tc)  # → '{"city": "Tokyo", "temp_c": 22}'
```

Secret providers

`kneo_agent.utils` exposes a small `SecretProvider` family for sourcing credentials (API keys, tokens) without hardcoding them — pass the resolved value into a runtime/provider factory. All providers share one method.

SecretProvider PROTOCOL

Structural protocol implemented by every provider below.

```
def get(name: str, *, default: str | None = None) → str
```

Return the secret for `name`. Raises `SecretNotFound` when it is absent and no `default` is given.

PROVIDER	CONSTRUCTOR	DESCRIPTION
<code>EnvSecretProvider</code>	<code>EnvSecretProvider(*, prefix="")</code>	Reads from environment variables. The logical name is upper-cased and the optional <code>prefix</code> prepended (e.g. <code>prefix="MYAPP_"</code> → <code>get("api_token")</code> reads <code>MYAPP_API_TOKEN</code>).
<code>FileSecretProvider</code>	<code>FileSecretProvider(base_path, *, encoding="utf-8", strip_whitespace=True)</code>	Reads each secret from a file under <code>base_path</code> . Name resolution is confined to the base dir — traversal (<code>..</code> , absolute paths, escaping symlinks) raises <code>ValueError</code> .
<code>MappingSecretProvider</code>	<code>MappingSecretProvider(secrets)</code>	Reads from an in-memory <code>Mapping[str, str]</code> — handy for tests.

`SecretNotFound` (a `KeyError` subclass) is raised by `get(...)` when a secret is missing and no default is supplied.

MCP

`kneo_agent.mcp` exposes lightweight MCP helpers for importing remote tools into the existing tool and skill pipeline.

MCPServerConfig DATACLASS

Connection description for one MCP server. Supported transports are `stdio`, `http`, and `sse`.

```
classmethod stdio(*, name, command, args = (), env = {}, cwd = None) → MCPServerConfig
```

```
classmethod http(*, name, url, headers = {}, timeout = 30.0, sse_url = None, max_response_bytes = 10485760, sse_read_timeout = None, verify = True, ca_bundle = None, client_cert = None, client_key = None) → MCPServerConfig
```

```
classmethod sse(*, name, sse_url, message_url = None, headers = {}, timeout = 30.0, max_response_bytes = 10485760, sse_read_timeout = None, verify = True, ca_bundle = None, client_cert = None, client_key = None) → MCPServerConfig
```

FIELD (HTTP / SSE)	DEFAULT	DESCRIPTION
<code>max_response_bytes</code>	10 MiB	Cap on bytes read from one server response (HTTP body / accumulated SSE event) — over the cap aborts the read rather than exhausting memory.
<code>sse_read_timeout</code>	None	Inactivity timeout (s) for an established SSE stream; None falls back to <code>timeout</code> . Bounds idleness so a silent server can't hang the reader.

<code>verify</code>	<code>True</code>	Verify the server's TLS certificate. Set <code>False</code> only for dev against a known-bad self-signed cert.
<code>ca_bundle</code>	<code>None</code>	Path to a PEM CA bundle for corporate / internal CAs.
<code>client_cert</code> / <code>client_key</code>	<code>None</code>	PEM client certificate + key for mTLS.

MCPClientSession CLIENT

Async MCP client session used by `ToolRegistry.register_mcp_server(...)`. Supports initialization, `tools/list`, and `tools/call`.

MCPTool DATACLASS

Lightweight MCP tool description with `name`, `description`, and `input_schema`, plus conversion to `ToolDefinition`.

Guide: See [MCP Guide](#) below and the runnable examples `examples/12_mcp_stdio_filesystem.py` and `examples/13_mcp_http_sse.py`.

messages utilities

`kneo_agent.utils.messages` — Convenience constructors and inspection helpers for `Message` lists.

Constructors

`user(content)``assistant(content)``system(content)`

`tool_result(content, *, tool_call_id, name=None)`

Inspection helpers

`last_user_message(messages) → Message | None`

`last_assistant_message(messages) → Message | None`

`filter_by_role(messages, role) → list[Message]`

Serialisation

`to_dict_list(messages) → list[dict]`

`from_dict_list(data) → list[Message]`

```
from kneo_agent.utils.messages import (
    user, assistant, system, tool_result,
    last_user_message, to_dict_list, from_dict_list,
)
import json

# Build a history from scratch
history = [
    system("You are a helpful assistant."),
    user("What is the capital of France?"),
    assistant("Paris."),
]

# Serialise to JSON for storage
serialised = json.dumps(to_dict_list(history))

# Restore from JSON
loaded = from_dict_list(json.loads(serialised))
agent.inject_history(loaded)

# Inspect
last = last_user_message(history)
print(last.content)  # "What is the capital of France?"
```

logging utilities

`kneo_agent.utils.logging` — Helpers for the `kneo_agent.*` logger hierarchy. Kneo Agent never adds handlers automatically; you opt in by calling `configure_logging()`.

```
configure_logging(level: str | int = "INFO", stream = sys.stderr) → None
```

Add a `StreamHandler` to the root `kneo_agent` logger. Idempotent — calling twice does not add duplicate handlers.

```
get_logger(name: str) → logging.Logger
```

Return a child logger named `"kneo_agent.<name>"`.

```
from kneo_agent.utils import configure_logging, get_logger

# Enable debug output for all kneo_agent internals
configure_logging("DEBUG")

# Get a namespaced logger for your own code
log = get_logger("my_app")
log.info("Starting agent session.")

# Or control via standard logging
import logging
logging.getLogger("kneo_agent").setLevel(logging.WARNING)
```

Exceptions

KneoAgentError (Exception)

- ├ **ConfigurationError**
- | └ **RuntimeNotConfiguredError**
- ├ **StreamingNotSupportedError**
- ├ **ToolNotFoundError**
- ├ **MaxIterationsReachedError**
- └ **ProviderError**

EXCEPTION	WHEN RAISED	NOTABLE ATTRIBUTES
-----------	-------------	--------------------

KneoAgentError	Base. Catch this to handle all SDK errors.	—
ConfigurationError	Agent or runtime is misconfigured.	—
RuntimeNotConfiguredError	AgentBuilder.build() called without a runtime.	—
StreamingNotSupportedError	agent.stream() called on a non-streaming runtime.	—
ToolNotFoundError	Model requested an unregistered tool.	.tool_name: str
MaxIterationsReachedError	Raised optionally; executors also signal via metadata.	—
ProviderError	Wraps errors from an underlying LLM provider.	.provider: str, .cause: Exception

```

from kneo_agent import KneoAgentError, StreamingNotSupportedError, ProviderError

try:
    result = await agent.run("Hello")
except StreamingNotSupportedError as e:
    print(f"Runtime does not stream: {e}")
except ProviderError as e:
    print(f"Provider '{e.provider}' failed: {e}")
    if e.cause:
        raise e.cause
except KneoAgentError as e:
    print(f"SDK error: {e}")

```

Writing a Custom Provider

Implement the `RuntimeImpl` protocol to integrate any LLM platform not natively supported. Your class only needs to satisfy the structural protocol — no inheritance required.

```

from typing import AsyncGenerator
from kneo_agent import Message, RunConfig, ToolCall
from kneo_agent.patterns import BridgeAgentFactory

class MyCustomImpl:
    platform_name = "my-platform"

    def __init__(self, my_client):
        self._client = my_client

    async def complete(
        self, messages: list[Message], config: RunConfig
    ) -> tuple[str, list[ToolCall]]:
        # Translate messages → your platform's format
        response = await self._client.generate(
            prompt=messages[-1].content,
            max_tokens=1024,
        )
        return response.text, [] # (text, tool_calls)

    async def execute_tool(self, call: ToolCall, config: RunConfig) -> str:
        # Route tool calls to registered handlers
        handler = config.extra.get("tool_handlers", {}).get(call.name)
        if handler:
            return handler(call.arguments)
        return f'{{"error": "tool {call.name!r} not registered"}}'

    async def stream_tokens(
        self, messages: list[Message], config: RunConfig
    ) -> AsyncGenerator[str, None]:
        async for token in self._client.stream(messages[-1].content):
            yield token

    def supports_tools(self) -> bool: return True
    def supports_streaming(self) -> bool: return True

# Wire it up via BridgeAgentFactory.custom()

```

```
runtime = BridgeAgentFactory.custom(MyCustomImpl(my_client), strategy="react")
agent = AgentBuilder().use_bridge(runtime).build()
```

Streaming Guide

All streaming runtimes yield `StreamChunk` objects with four possible types. The stream always ends with `type == "done"`.

```

import sys

async def stream_with_tool_trace(agent, message: str):
    collected_text = []
    tool_calls = []

    async for chunk in await agent.stream(message):
        match chunk.type:
            case "text":
                print(chunk.content, end="", flush=True)
                collected_text.append(chunk.content)

            case "tool_call":
                tc = chunk.tool_call
                print(f"\n[→ {tc.name}({tc.arguments})]")
                tool_calls.append(tc)

            case "tool_result":
                tr = chunk.tool_result
                print(f"[- {tr.name}: {tr.result[:60]}]")

            case "done":
                print() # final newline

    return "".join(collected_text), tool_calls

text, tools = await stream_with_tool_trace(agent, "Research quantum computing.")

```

Note: `agent.stream()` is itself `async` (because it may check streaming support before yielding), so always use `await` on the call and `async for` on the result.

Bridge vs Adapter — Decision Reference

Dimension	Bridge	Native	Adapter
Intent	Decouple strategy from platform so both can vary independently	Let the platform SDK/runtime own the loop directly	Convert an existing, fixed interface into the target protocol
When to use	Designing from scratch; need Kneo-owned strategies and a Bridge-compatible provider	Platform has a strong native runtime you want to preserve	Existing platform object already running; cannot change its interface
Strategy control	You own the loop (Simple / ReAct / Plan-Act)	Platform owns the loop; strategy is metadata or platform-specific behavior	Platform owns the loop; no strategy choice
Class count	$M + N$ (not $M \times N$): $3 + 3 = 6$ classes for 9 variants	One runtime per native platform family	Exactly N classes (one per platform)
Runtime name	"react@langchain"	"react@openai-agents" or "google-adk-native"	"openai-agents-adapter"
Extensibility	New strategy → one class; works on all platforms automatically	New native runtime → one platform-owned implementation	New platform → one new adapter; no impact on others
Migration path	Use for Kneo-managed strategies	Use for OpenAI Agents SDK and native Google ADK execution	Use when migrating existing agents incrementally
Factory	BridgeAgentFactory.for_*	NativeRuntimeFactory.for_*	AdapterAgentFactory.for_*
Builder method	.use_bridge(runtime)	.use_runtime(runtime)	.use_adapter(runtime)

Guides

These guide sections are embedded directly in the API reference so the reference manual reads as a single document instead of sending you to separate files.

Agent Middleware Guide GUIDE

Middleware is the SDK's cross-cutting interception layer. It is best for logging, guardrails, request mutation, tool-result rewriting, and short-circuiting.

`wrap_run(...)` and `wrap_stream(...)` apply to every runtime style.

`wrap_model_call(...)` and `wrap_tool_call(...)` apply only to Bridge runtimes, where Kneo owns the inner loop.

Static middleware comes from `AgentBuilder.add_middleware(...)` or `with_middlewares(...)`.

Per-run middleware comes from `RunConfig(middlewares=[...])`.

```
from kneo_agent import AgentBuilder, BaseAgentMiddleware, RunConfig

class LoggingMiddleware(BaseAgentMiddleware):
    async def wrap_run(self, context, handler):
        print(f"running {context.agent_name}")
        return await handler(context)

agent = (
    AgentBuilder()
    .add_middleware(LoggingMiddleware())
    .use_runtime(runtime)
    .build()
)

result = await agent.run(
    "hello",
    run_config=RunConfig(middlewares=[LoggingMiddleware()]),
)
```

Examples: See `examples/16_agent_middleware_logging.py` and

`examples/17_agent_middleware_short_circuit.py` .

Agent Skills Guide GUIDE

Kneo Agent supports both in-memory skills and disk-backed Agent Skills-compatible `SKILL.md` directories.

The Markdown body becomes `Skill.system_prompt` .

Frontmatter `tools` becomes `ToolDefinition[]` .

`defaults` contributes to `RunConfig` .

`extra` merges into `RunConfig.extra` .

Middleware is intentionally separate: skills compile into data, middleware stays executable policy.

```
from kneo_agent import load_skill

skill = load_skill("examples/skills/weather")
skill = skill.with_tool_handlers({"get_weather": get_weather})

print(skill.resource_paths())
print(skill.read_resource("references/REFERENCE.md"))
```

Progressive disclosure: bundled `references/` , `scripts/` , and `assets/` stay discoverable but are not injected automatically into every run.

MCP Guide GUIDE

MCP is treated as an external tool source, not a separate runtime family.

Connect with `MCPServerConfig` .

Discover tools with `ToolRegistry.register_mcp_server(...)` .

Convert them into ordinary `ToolDefinition` plus proxy handlers.

Attach the registry with `AgentBuilder.with_tool_registry(...)` .

Bridge, Native, and Adapter runtimes can all use MCP tools through the existing `RunConfig` and `tool_handlers` path.

```

from kneo_agent import MCPServerConfig
from kneo_agent.utils import ToolRegistry

registry = ToolRegistry()
await registry.register_mcp_server(
    MCPServerConfig.stdio(
        name="filesystem",
        command="npx",
        args=["-y", "@modelcontextprotocol/server-filesystem", "/tmp"],
    ),
    prefix="fs_",
)

```

Middleware pairing: use run middleware for tracing or guardrails, and Bridge tool-call middleware when you want to inspect or rewrite MCP tool results before the next model turn.

Human In The Loop Guide GUIDE

Human review is modeled as a normal workflow participant rather than a separate runtime family.

Use middleware for automated policy, logging, and short-circuiting.

Use `WorkflowBuilder.human_step(...)` when a real person must approve, edit, or pause execution.

Human steps work in sequential workflows, graph workflows, nested workflows, and orchestration runtimes.

```

from kneo_agent.workflows import WorkflowBuilder

review = WorkflowBuilder.human_step(
    "approval",
    "Approve this draft?",
    resolver=lambda request: "Approved by Alice",
)

```

Examples: See `examples/14_workflow_human_in_loop_resolver.py` and `examples/15_workflow_human_in_loop_pause.py`.

Kneo Agent v2.0.0 · MIT License · Python 3.12+ · `pip install kneo-agent`

Run the test suite: `pytest -q tests` · Or: `make test`